ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл холбооны технологийн сургууль



БИЕ ДААЛТЫН АЖЛЫН MANUAL (ЗААВАР)

Үйлдлийн систем (F.CSM302)

2026-2027 оны хичээлийн жилийн намар

Бие даалтын ажлын нэр: The Dining-Philosophers Problem — Бие даалт 1 (хуваарилалтын код II-4-11)

Хичээл заасан багш: Ганбат

Лабораторийн ажил гүйцэтгэсэн: Оюутан: Г. Сэлэнгэ (B242270106)

Оюутан: Ч. Гантогтох (B242270005)

Оюутан: Э. Энхсаруул (B232270145)

Оюутан: Э. Төгөлдөр (B241940019)

Улаанбаатар хот
2026 он

1. ХЭРЭГЛЭГЧИЙН ЗААВАР

Програмын зорилго

Энэ програм сурах бичгийн (Operating System Concepts, 7-р бүлэг) Dining-Philosophers Problem-ийг хэрэгжүүлж, хэрэглэгчид philosopher-ийн тоо болон бодох/идэх хугацааны мужийг өөрчлөх, deadlock-гүй (monitor) болон deadlock-той (naive) хоёр алгоритмыг харьцуулах, philosopher бүрийн thinking → hungry → eating төлөв шилжилт болон fork (нөөц) эзэмшилтийг бодит цагийн state-diagram UI-гаар ажиглах боломжийг олгодог.

Дэмжих орчин

- ОС: Linux (Ubuntu 22.04/24.04 дээр туршигдсан), WSL2 дээр мөн ажиллана
- Compiler: GCC ≥ 9 (-std=gnu11, pthread)
- Python ≥ 3.8 (зөвхөн стандарт сан — UI сервер, шалгалтын скрипт)
- Browser: орчин үеийн ямар ч браузер (Chrome, Firefox, Edge) — зөвхөн SVG + fetch streaming ашигладаг, гадны JS сан татдаггүй

Суулгах эсвэл нээх алхам

```
git clone https://github.com/Selenge555/-FCSA_302_II-4-11-dining-philosophers.git
cd ./-FCSA_302_II-4-11-dining-philosophers
make all        # build/dp үүсгэнэ
make serve      # http://127.0.0.1:8000/ дээр сервер эхэлнэ
```

Хөтчөөр http://127.0.0.1:8000/ хаягийг нээнэ. UI нь 5 алхамтай: 1) Танилцуулга, 2) Дүрэм, 3) Deadlock, 4) Шийдэл, 5) Бодит туршилт. Доод талын «Дараагийн →/← Буцах» товч, дээд талын табууд эсвэл ←/→ товчоор шилжинэ. Эхний дөрвөн дэлгэц нь асуудлыг жижиг загвараар алхам алхмаар тайлбарлана (philosopher дээр дарж төлөв шилжүүлэх, Алхам ▶️ товчоор deadlock үүсгэх, шийдлийн тайлбар); доорх бүх тохиргоо, ▶️ Эхлүүлэх, Replay зэрэг нь 5-р дэлгэц «Бодит туршилт» дээр байна (шууд нээх: http://127.0.0.1:8000/#real).

Оролт ба тохиргоо

Хүснэгт 1.1 — Тохируулах талбарууд

Талбар	Утга	Хязгаар
Алгоритм	monitor (deadlock-гүй) эсвэл naive (deadlock-той)	—
Philosopher-ийн тоо (n)	—	2–64
Бодох хугацаа (min/max, мс)	—	0–600000
Идэх хугацаа (min/max, мс)	—	0–600000
Ажиллах хугацаа (мс)	—	100–600000
Seed	бодох/идэх хугацааны санамсаргүй дарааллыг л тогтооно (OS scheduling-ийг биш — доорх тодруулгыг үзнэ үү)	0–2000000000

Талбар	Утга	Хязгаар
Fork авах завсар (зөвхөн naive, мс)	зүүн/баруун fork авах хоорондын хугацаа (deadlock-ыг нотлоход)	0–10000

Бүх талбарыг UI дээрх 5 бэлэн preset товчлууруар (Сурах бичиг / Хурдан demo / 12 philosopher / Deadlock demo / Starvation demo) шууд дуудаж болно.

Seed тухай тодруулга: --seed нь зөвхөн бодох/идэх хугацааны санамсаргүй дарааллыг детерминистээр тогтооно — thread-үүдийг ажиллуулах OS-ийн scheduling дарааллыг тогтоодоггүй. Иймд monitor алгоритмыг ижил seed-ээр хэд хэдэн удаа ажиллуулахад philosopher бүрийн идсэн хоолны тоо (meals) ажиллагаа бүрт бага зэрэг өөр гарч болно — энэ нь алдаа биш, OS-ийн thread scheduling-ийн байгалийн хувьсамтгай чанараас шалтгаална (дэлгэрэнгүйг Report.docx-ийн 8-р хэсгээс үзнэ үү).

Ажиллуулах, зогсоох, reset хийх

- ▶️ Эхлүүлэх — сонгосон тохиргооноос build/dp-г серверээр дамжуулан ажиллуулж, гарсан event-ийг бодит цагт SVG дээр зурна
- ■ Зогсоох — ажиллаж буй процессыг зогсооно (клиент холболтоо тасалсныг сервер 1 секунд тутамд шалгадаг тул бодит туршилтаар хамгийн ихдээ ~2 секундын дотор илэрч, dp процессыг kill хийдэг — TCP холболт заримдаа нэг ping бичилтийг "амжилттай" авч, зөвхөн ХОЁР дахь бичилт дээр алдаа буцаадаг тул яг 1 секунд биш, 2 ping хүртэл үргэлжилж болно; тест: tests/run_tests.sh-ийн "Stop (холболт тасрах) дараа slot чөлөөлөгдөж..." регрессийн шалгалт)
- ⟲ Reset — одоогийн тохиргоо, event лог, графикийг цэвэрлэж эхний төлөвт буцаана
- Replay (◀️ / слайдер / ▶️ / Live; бодит туршилтын дэлгэц дээр ←/→ товч мөн ажиллана) — ажиллагаа дууссаны дараа event бүрийг алхам алхмаар эргүүлж харах боломжтой
- ▶️▶️ N = 5, 12, 32, 64 цуврал — одоогийн бодох/идэх/хугацааны тохиргоогоор philosopher-ийн тоог 5, 12, 32, 64 болгон дөрвөн ажиллагааг дараалан ажиллуулж, бүгдийг туршилтын түүхэд хадгална (■ Зогсоох дарвал цуврал зогсоно)
- «Удаан хүлээлт — starvation-ийн эрсдэл» карт — анхааруулах босгоос (анхдагч нь max(500мс, 4 × идэх max); «авто»-г арилгаад гараар өөрчилж болно) удаан hungry байгаа philosopher-ийг жагсааж, ширээн дээр хүрэн тасархай хүрээгээр тэмдэглэнэ. Энэ нь starvation-ийн ЭРСДЭЛ бөгөөд батлагдсан starvation биш — картан дээрх тайлбарыг үзнэ үү. Replay үед ч тухайн мөчийн анхааруулга харагдана

- Статистик — ажиллагаа дууссаны дараа ⚠️ багана (явцын дунд босгоос удаан хүлээсэн philosopher), мөн Jain-ийн индекс (1 = бүгд тэнцүү идсэн), min/max хоолны харьцаа, дундаж ба хамгийн их хүлээлтийн хураангуй гарна
- «Туршилтын түүх ба харьцуулалт» — дууссан ажиллагаа бүр хөтөчид (localStorage) хадгалагдаж, хүснэгт болон philosopher-ийн тооноос (N) хамаарах график гарна (үзүүлэлтийг сонгоно; нэг шугам = ижил алгоритм ба тохиргоо). CSV/JSON татах, JSON оруулах (жишээ нь tools/experiment.py-ийн үр дүн), ✕ дарж нэг бүрчлэн устгах боломжтой

Дүрслэлийн тэмдэг, өнгө, төлөв

Хүснэгт 1.2 — Өнгөний тэмдэглэгээ

Өнгө	**Утга**
Цэнхэр	thinking — бодож байна
Шар-улбар	hungry — fork хүлээж байна
Ногоон	eating — хоёр fork-той, идэж байна
Саарал шугам	fork чөлөөтэй (ширээн дээр)

Fork нь чөлөөтэй үед ширээний голд, эзэнтэй үед эзэмшигч philosopher рүүгээ ойртуулж, эзэмшигчийнхээ өнгөөр зурагдана. Deadlock илэрвэл philosopher улаан хүрээтэй болж, баннераар "DEADLOCK" гэж мэдэгдэнэ. Ажиллагаа дууссаны дараа ч deadlock-ийн мөчийн зураг (fork бүр эзэмшигчийн өнгөөр) хэвээр харагдана.

Жишээ 1 — Сурах бичгийн монитор шийдэл (deadlock-гүй)

1. Preset: Сурах бичиг (monitor, n=5, бодох/идэх 1000–3000мс, 20с)

2. ▶️ Эхлүүлэх дарна.

3. Ажиглах зүйл: philosopher бүр тогтмол thinking → hungry → eating мөчлөгөөр эргэлдэж, зэргэлдээх хоёр philosopher хэзээ ч зэрэг eating болохгүй. Статистикийн хүснэгтэд ихэнх тохиолдолд philosopher бүр ойролцоогоор ижил тооны хоол идсэн байхыг ажиглаж болно — гэхдээ энэ нь monitor алгоритмын баталгаажуулсан шинж чанар (fairness guarantee) БИШ, зөвхөн энэ хэрэгжилтийн богино хугацааны ажиглалт (доор "Known issues" ба Report.docx-ийн 3, 9-р хэсгийг үз).

Жишээ 2 — Naive шийдэл дэх deadlock

1. Preset: Deadlock demo (naive, n=5, бодохгүй, идэх 300–600мс, fork-delay 300мс)

2. ▶️ Эхлүүлэх дарна.

3. Ажиглах зүйл: philosopher бүр зүүн fork-оо авч, баруун fork-оо хүлээж байх зуур, бүгд зэрэг hungry төлөвт царцана → улаан хүрээ, "DEADLOCK" баннер гарч ажиллагаа reason:"deadlock"-оор дуусна.

Жишээ 3 — Monitor шийдэл дэх starvation-ийн эрсдэл

1. Preset: Starvation demo (monitor, n=12, бодох 0, идэх 10–50мс, 10с)

2. ▶️ Эхлүүлэх дарна.

3. Ажиглах зүйл: хэдэн секундын дотор philosopher-уудын хагас нь (сондгой эсвэл тэгш дугаартай нь ээлжлэн, жишээ нь P1, P3, P5, …) hungry хэвээр үлдэж, ширээн дээр хүрэн тасархай хүрээ, «Удаан хүлээлт» картад ⚠️ гарна; дууссаны дараа статистикт Jain ≈ 0.5, min/max ≈ 0 харагдана (бодит 10с-ийн ажиллагаанд seed 1–3: Jain = 0.50–0.51). Philosopher-ийн тоог 13 (сондгой) болгоод дахин ажиллуулбал хуваарилалт шударга болно (тайлбарыг Report-ийн 8–9-р хэсгээс үзнэ үү).

Жишээ 4 — Philosopher-ийн тоогоор харьцуулах

1. Preset: Хурдан demo (эсвэл хүссэн бодох/идэх хугацаа), хугацааг 5000–10000мс болгоно.

2. ▶️▶️ N = 5, 12, 32, 64 цуврал дарж, дөрвөн ажиллагаа дуусахыг хүлээнэ.

3. «Туршилтын түүх ба харьцуулалт» хэсгийн «Үзүүлэлт»-ээс хооллолт, хүлээлт, Jain зэргийг сонгож график дээр N-ээс хэрхэн хамаарахыг харна; CSV татаж Excel-д нээж болно.

Түгээмэл алдаа ба шийдэл

Хүснэгт 1.3 — Алдаа ба шийдэл

Алдаа	Шалтгаан	Шийдэл
"Серверт холбогдож чадсангүй"	ui/server.py ажиллаагүй	make serve ажиллуулах
"build/dp олдсонгүй"	compile хийгдээгүй	make all ажиллуулах
"Philosopher-ийн тоо: 2..64 хооронд"	n талбарт хүчингүй утга	2–64-ийн хооронд тоо оруулах
"Бодох доод хугацаа дээд хугацаанаас их"	min > max	min ≤ max болгож засах
UI дээр юу ч зурагдахгүй	Start дарахаасаа өмнө тохиргоо алдаатай	Улаан алдааны жагсаалтыг уншиж засах
Туршилтын түүх дахин нээхэд алга болсон	Хөтчийн private/incognito горимд localStorage хадгалагддаггүй	Ердийн цонхонд нээх; эсвэл «JSON татах»-аар хадгалаад дараа нь «JSON оруулах»

2. OPERATIONAL ЗААВАР

Repository clone хийх

```
git clone https://github.com/Selenge555/-FCSA_302_II-4-11-dining-philosophers.git
cd ./-FCSA_302_II-4-11-dining-philosophers
```

ОС, compiler/runtime, хувилбар

- Linux (Ubuntu 22.04+ эсвэл WSL2 Ubuntu)
- gcc --version ≥ 9 (-std=gnu11 дэмждэг байх)
- python3 --version ≥ 3.8
- (Сонголтоор) node --version ≥ 14 — tests/test_model.js-г ажиллуулахад
- (Сонголтоор) valgrind, thread sanitizer дэмжлэгтэй gcc/clang — make memcheck, make tsan

Dependency суулгах

Гадны сангийн dependency байхгүй (зөвхөн стандарт C/POSIX болон Python стандарт сан). Сонголтоор:

```
sudo apt install valgrind    # make memcheck-д хэрэгтэй
```

Build ба Run команд

```
make all                     # build/dp
make serve                   # UI сервер (http://127.0.0.1:8000/)
make test                    # бүх автомат шалгалт
make tsan                    # ThreadSanitizer хувилбар (build/dp_tsan)
make memcheck                # valgrind helgrind + leak-check
make clean                   # цэвэрлэх
python3 tools/experiment.py --dur 60000 --jobs 2 \
    --csv out.csv --json out.json   # урт хугацааны fairness туршилт (UI-д JSON оруулна)
```

Тушаалын мөрөөс шууд:

```
./build/dp -a monitor -n 5 --think-min 1000 --think-max 3000 \
           --eat-min 1000 --eat-max 3000 -d 10000 -s 1
./build/dp -a naive -n 5 --think-min 0 --think-max 0 --eat-min 300 --eat-max 600 \
           --grab-delay 300 -d 8000 -s 1   # deadlock-г нотолно (exit code 3)
```

Үндсэн тестийн өгөгдөл

tests/examples/ дотор 4 жишээ .jsonl гаралт (давтагдах боломжтой seed-тэй):

Хүснэгт 2.1 — Жишээ лог файлууд

Файл	Тохиргоо	Хүлээгдэх үр дүн
monitor_n5_sample.jsonl	monitor, n=5, seed=1	timeout, бүгд ≥3 удаа идсэн
naive_deadlock_n5_sample.jsonl	naive, n=5, seed=1, grab-delay=300	deadlock, 0 хоол
monitor_n2_sample.jsonl	monitor, n=2 (доод хязгаар), seed=2	timeout, инвариант зөрчигдөхгүй
monitor_n64_sample.jsonl	monitor, n=64 (дээд хязгаар), seed=3	timeout, инвариант зөрчигдөхгүй

Шалгах:

```
python3 tests/check_log.py tests/examples/monitor_n5_sample.jsonl
python3 tests/check_log.py --expect-deadlock tests/examples/naive_deadlock_n5_sample.jsonl
```

Нэмэлт boundary/error test

```
./build/dp -n 1                          # exit 2: n доод хязгаараас гадуур
./build/dp -n 5 --think-min 100 --think-max 50   # exit 2: min > max
./build/dp -n 65                          # exit 2: n дээд хязгаараас гадуур (MAX_PHIL=64)
```

make test энэ бүгдийг автоматаар шалгадаг.

Хүлээгдсэн үр дүн

- monitor алгоритм дээр ажиллах хугацаанд хэзээ ч deadlock, mutual-exclusion зөрчил гарахгүй; харин starvation-аас хамгаалдаггүй: бодох хугацаа 0 ба philosopher-ийн тоо тэгш үед хагас нь хэдэн арван секунд иддэггүй үе гардаг (Жишээ 3, Report-ийн 8–9-р хэсэг). Бодох хугацаа > 0 эсвэл тоо сондгой үед хуваарилалт шударга (Jain ≈ 0.99–1)
- naive алгоритм дээр, бодох хугацаа 0, бүх philosopher ижил хугацаанд эхэлж (жишээ нь grab-delay>0-той) зүүн fork-оо авбал, deadlock детерминистээр давтагдана (гарах код 3); харин grab-delay=0 үед эсрэгээр deadlock биш, ноцтой starvation ажиглагдсан (9-р хэсэг, Report.docx)
- make test бүх 23 шалгалт OK, make memcheck дөрвөн valgrind команд (monitor, naive тус бүрд helgrind + leak-check) алдаагүй (хоосон гаралт)

Known issues

- naive алгоритмын deadlock нь судалгааны зорилготой учир зассангүй (энэ бол алдаа биш, сурах бичгийн "naive" жишээг нотлох зориудын зан төлөв)
- UI сервер зэрэгцээ ажиллагааг 4-өөр хязгаарласан — олон tab нэгэн зэрэг Start дарвал сүүлчийнх нь 429 алдаа авч болзошгүй